

FRA503 Deep Reinforcement Learning for Robotics

Homework 3: Cartpole Function Approximation Algorithms

Name	Student ID
Chantouch Orungrote	66340500011
Sasish Kaewsing	66340500076

1 CartPole Problem

The CartPole stabilization task requires applying continuous forces to keep a hinged pole upright. Failure occurs if the pole angle exceeds $\pm 28.6^\circ$ or the cart leaves the track; success requires surviving 500 steps. The Isaac Lab track boundary is wider than the classical ± 2.4 m Gymnasium threshold, retained as an approximate reference in plots.

1.1 Mathematical Formulation

Concept	Form	Definition
Value Function	$\hat{v}(s, \mathbf{w})$	State-value function approximated by weight vector $\mathbf{w} \in \mathbb{R}^d$, replacing the lookup table.
Q-Function	$\hat{q}(s, a, \mathbf{w})$	Action-value function approximated by a parameterized model, enabling generalization across continuous states.
Policy Gradient	$\nabla_{\boldsymbol{\theta}} J = \mathbb{E}_{\pi} [\sum_t \gamma^t q_{\pi}(S_t, A_t) \nabla_{\boldsymbol{\theta}} \ln \pi(A_t S_t, \boldsymbol{\theta})]$	Exact gradient of the performance objective w.r.t. policy parameters, without requiring derivatives of the state distribution.
Advantage	$A_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s)$	How much better action a is than the average under π ; used as a variance-reducing baseline.
TD Error	$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w})$	One-step TD error used by actor-critic methods as a low-variance estimate of the advantage.
GAE	$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma \lambda)^l \delta_{t+l}$	Generalized Advantage Estimation; λ controls the bias-variance tradeoff.

Table 1: Mathematical formulation of function approximation-based RL.

2 Function Approximation Algorithms

Algorithm	Type	Policy	On/Off	Actions	Exploration
Linear_QN	Value	Deterministic	Off	Discrete	ε -greedy decay
DQN	Value	Deterministic	Off	Discrete	ε -greedy decay
MC REINFORCE	Policy	Stochastic	On	Discrete	Softmax stochasticity
AC	Actor-Critic	Stochastic	On	Continuous	Gaussian noise (learned σ)
A2C	Actor-Critic	Stochastic	On	Continuous	Gaussian noise + entropy bonus
PPO	Actor-Critic	Stochastic	On	Continuous	Gaussian noise + entropy bonus
TD3	Actor-Critic	Deterministic	Off	Continuous	Additive Gaussian noise
SAC	Actor-Critic	Stochastic	Off	Continuous	Max-entropy objective

Table 2: Properties of all implemented algorithms.

Linear Q-Learning uses a linear action-value function $\hat{q}(s, a, \mathbf{w}) = \mathbf{w}_a^\top \mathbf{x}(s)$ with semi-gradient Q-learning updates:

$$\mathbf{w}_a \leftarrow \mathbf{w}_a + \alpha \left[R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a', \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w}) \right] \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w}) \quad (1)$$

DQN replaces the linear model with a deep neural network. Three stabilization mechanisms—*experience replay*, a *target network* $\tilde{q}$, and *gradient clipping*—address temporal correlation and bootstrapping instability:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[R_{t+1} + \gamma \max_{a'} \tilde{q}(S_{t+1}, a', \mathbf{w}^-) - \hat{q}(S_t, A_t, \mathbf{w}) \right] \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w}) \quad (2)$$

MC REINFORCE parameterizes the policy $\pi(a|s, \boldsymbol{\theta})$ with a softmax network and updates via the full episodic return:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \sum_{t=0}^T \gamma^t G_t \nabla_{\boldsymbol{\theta}} \ln \pi(A_t | S_t, \boldsymbol{\theta}) \quad (3)$$

Actor-Critic (AC) introduces a critic $\hat{v}(s, \mathbf{w})$ to reduce variance via the one-step TD error:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \delta_t \nabla_{\boldsymbol{\theta}} \ln \pi(A_t | S_t, \boldsymbol{\theta}), \quad \mathbf{w} \leftarrow \mathbf{w} + \alpha_w \delta_t \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) \quad (4)$$

Advantage Actor-Critic (A2C) extends AC with GAE $\hat{A}_t = \sum_l (\gamma \lambda)^l \delta_{t+l}$ and a combined loss:

$$\mathcal{L} = -\mathbb{E} \left[\hat{A}_t \ln \pi(A_t | S_t) \right] + c_v \mathbb{E} [(G_t - \hat{v}(S_t))^2] - c_e \mathcal{H}[\pi(\cdot | S_t)] \quad (5)$$

Proximal Policy Optimization (PPO) adds a *clipped surrogate objective* over A2C to prevent destructive policy updates, where $r_t = \pi_{\boldsymbol{\theta}} / \pi_{\boldsymbol{\theta}_{\text{old}}}$:

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E} \left[\min \left(r_t(\boldsymbol{\theta}) \hat{A}_t, \text{clip}(r_t(\boldsymbol{\theta}), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right] \quad (6)$$

Twin Delayed DDPG (TD3) addresses Q-overestimation via *twin critics* (target uses min), *delayed actor updates* (every 2 critic steps), and *target policy smoothing*:

$$y = R_{t+1} + \gamma \min_{i=1,2} Q_{\mathbf{w}_i^-}(S_{t+1}, \tilde{a}), \quad \tilde{a} = \mu_{\theta^-}(S_{t+1}) + \text{clip}(\epsilon, -c, c) \quad (7)$$

Soft Actor-Critic (SAC) maximizes expected return plus policy entropy, embedding exploration directly in the objective:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}[r(S_t, A_t) + \alpha \mathcal{H}(\pi(\cdot|S_t)))] \quad (8)$$

3 Experimental Design

All algorithms are trained on the CartPole stabilization task in Isaac Lab for 10,000 episodes (max 500 steps/episode). The observation space is $[x, \theta, \dot{x}, \dot{\theta}]$; the action space is a continuous force in $[-10, 10]$ N.

3.1 Hyperparameters

All algorithms share learning rate $\alpha = 3 \times 10^{-4}$ and discount $\gamma = 0.99$. Algorithm-specific parameters are in Table 3.

Algorithm	Key Parameters
Linear_QN	11 actions, $\alpha = 10^{-3}$, ϵ : 1.0 $\rightarrow$ 0.01 (decay 5×10^{-4})
DQN	11 actions, hidden 128, $\tau = 0.005$, buffer 50k, batch 128, target update / 200 steps
MC REINFORCE	11 actions, hidden 128, dropout 0.1
AC	1 cont. action, hidden 256, entropy coeff. 0.01
A2C	1 cont. action, hidden 256, $\lambda_{\text{GAE}} = 0.95$, $c_v = 0.5$, $c_e = 0.01$, 64 trans./rollout
PPO	1 cont. action, hidden 256, $\lambda_{\text{GAE}} = 0.95$, $\epsilon_{\text{clip}} = 0.2$, 4 epochs, 4 mini-batches, 64 trans./rollout
TD3	1 cont. action, hidden 256, $\tau = 0.005$, buffer 100k, batch 256, noise: 0.2/0.5/0.1, delay 2
SAC	1 cont. action, hidden 256, $\tau = 0.005$, buffer 100k, batch 256, $\alpha = 0.2$ (auto-tuned)

Table 3: Algorithm-specific hyperparameters.

3.2 Evaluation Metrics

- **Learning Efficiency:** Training reward curve, solved episode (first episode where 100-episode rolling average ≥ 450 steps), and rolling reward standard deviation (window = 100).
- **Deployment Performance:** 100 deterministic episodes reporting average reward, episode length, success rate, mean absolute pole angle $|\theta|$, pole angle standard deviation, cart position drift, and action smoothness.

4 Analysis

4.1 Training Performance Overview

The training results reveal a clear three-tier performance hierarchy as illustrated in Figure 1.

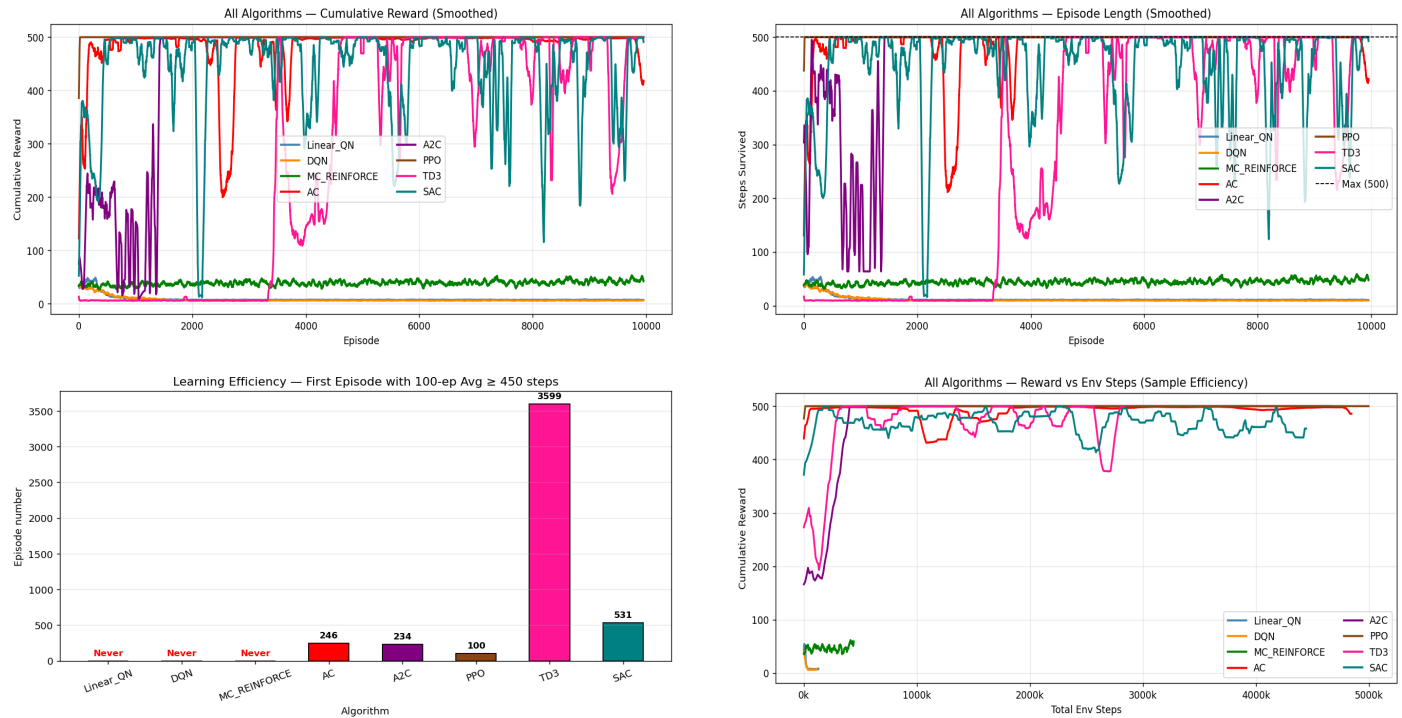


Figure 1: Training reward and episode length curves over 10,000 episodes top; solved-episode marker and steps-to-reward sample efficiency bottom.

Tier 1 — Failed to Converge: Linear_QN, DQN, and MC REINFORCE These algorithms **never** reached the 450-step threshold.

- **Linear_QN:** Stalled at approximately 40 reward. *Primary cause:* The linear model lacks the representational capacity to process non-linear dynamics, such as stabilizing a pole while the cart moves in the opposite direction.
- **DQN:** Reached 50 reward briefly before collapsing to near 10. *Primary cause:* Irreversible training collapse occurred due to extreme Q-value overestimation. The network assigned high values to suboptimal actions, ultimately destroying the learning process.
- **MC REINFORCE:** Fluctuated randomly between 20 and 50 reward. *Primary cause:* Excessive noise in the learning signals prevented any meaningful progress. Without a value critic to stabilize the updates, the policy updates were heavily distorted by random chance.

Tier 2 — Unstable Convergence: AC, TD3, and SAC These algorithms passed the threshold but demonstrated **delayed learning or persistent instability**.

- **AC:** Averaged near 500 reward but experienced sharp drops to around 100. *Primary cause:* The learned policy was brittle in rare states. It functioned perfectly under normal conditions but failed to recover when pushed slightly off balance.
- **TD3:** Remained below 250 reward until episode 3,500, followed by a sudden improvement. *Primary cause:* Off-policy methods require extensive data collection before the critic can provide accurate guidance to the actor.
- **SAC:** Learned quickly but frequently dropped to around 200 reward. *Primary cause:* Its entropy-maximizing objective forces continuous exploration. This mechanism actively sacrifices short-term stability to evaluate new actions.

Tier 3 — Robust Convergence: PPO and A2C

These algorithms achieved **rapid and stable mastery**.

- **PPO:** The most consistent and efficient algorithm. It reached 500 reward by episode 100 and maintained maximum performance. *Primary cause:* Its clipping mechanism acts as a strict constraint, ensuring that every policy update is progressive and safe.
- **A2C:** Reached 500 reward but suffered a **severe performance collapse** near episode 2,000 before eventually recovering. *Primary cause:* It lacks a protective clipping mechanism. A single batch of poor data caused a destructive update that temporarily compromised its learned abilities.

4.2 Training Stability

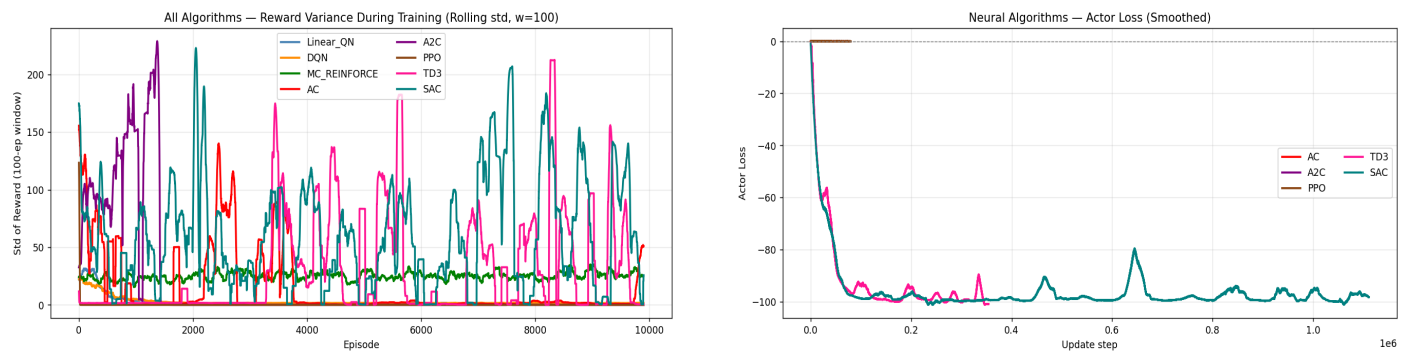


Figure 2: 100-episode rolling reward standard deviation left and actor/critic loss over training steps right.

The reward variance chart in Figure 2 highlights **the stability of the learning process**. **PPO’s variance drops to near-zero** almost immediately, proving that its clipped objective successfully prevents destructive updates. Conversely, **A2C exhibits the highest variance peak** reaching above 225, which visually confirms its catastrophic collapse. This failure occurs because a single flawed gradient step generates poor rollouts, feeding corrupted data into the next update and creating a destructive feedback loop. SAC maintains a persistent moderate variance throughout training because the algorithm is designed to continuously explore.

Analyzing the loss curves, **DQN presents a massive loss spike** reaching 2.3×10^7 . This provides clear evidence of the **Deadly Triad**—the fatal combination of off-policy learning, bootstrapping, and neural network approximation. Once DQN overestimates a Q-value, that error is permanently stored in the replay buffer. The network subsequently trains on its own corrupted data, causing the loss to explode and preventing any possible recovery.

4.3 The On-Policy vs. Off-Policy Trade-off

Algorithm	Solved Episode	Final Avg. Reward	Reward Variance
Linear_QN	Never	≈ 40	Low
DQN	Never	≈ 10	Low collapsed
MC REINFORCE	Never	≈ 35	High ≈ 25 std
AC	246	≈ 470	Moderate with drops
A2C	≈ 500	500	Near-zero post-recovery
PPO	100	500	Near-zero
SAC	531	≈ 460	Moderate
TD3	3,599	500	Low post-convergence

Table 4: Convergence summary highlighting the speed discrepancy between paradigms.

In theoretical reinforcement learning, Off-policy methods like TD3 and SAC are expected to learn faster by reusing historical data from a memory buffer. However, Table 4 presents a **significant empirical contradiction**: On-policy methods outperformed them by a massive margin. **PPO solved the task five times faster than SAC and thirty-five times faster than TD3.**

Why did this happen? In highly sensitive control tasks like CartPole, **learning from recent experience proved far more effective than reusing historical data.** Off-policy methods suffer from **distributional shift**—they attempt to evaluate current actions using outdated transitions from past policies. This severely slows down early training because the critic struggles to provide accurate evaluations. In contrast, On-policy methods strictly utilize fresh data generated by the current policy. When paired with Generalized Advantage Estimation, PPO and A2C receive highly accurate and immediately actionable feedback. This fresh gradient signal allows them to converge rapidly, entirely avoiding the early instability that hinders off-policy methods.

4.4 Why Did Discrete-Action Algorithms Fail?

- **Linear QN — Insufficient Capacity:** A linear model cannot compute complex interactions. Balancing a pole while the cart moves toward the track boundary requires a non-linear control strategy. Linear Q-learning lacks the architectural complexity required to process these combined variables.
- **DQN — Discrete Limitations and Divergence:** Dividing the continuous force into 11 discrete bins limits the agent to making large two-Newton adjustments. This coarse resolution is far too aggressive for delicate stabilization, forcing the agent to overcorrect constantly. Furthermore, the **Deadly Triad** caused its Q-values to diverge uncontrollably, permanently breaking the learning process.
- **MC REINFORCE — Excessive Gradient Variance:** Because this algorithm relies strictly on the total episodic reward without a value critic, every policy update is subjected to massive variance. The learning signal was simply too noisy for the neural network to isolate which specific actions were responsible for success or failure.

4.5 Why Did Actor-Critic Methods Succeed?

Actor-Critic methods succeeded because they output **continuous actions** without artificial discretization and utilize a **Critic** to provide reliable learning signals. The architectural progression from AC to PPO illustrates exactly how to build a robust control system:

- **AC — Fundamental Variance Reduction:** Introducing a Critic resolved the extreme variance problem of MC REINFORCE. However, AC relies on a basic one-step update that produced a fragile policy, resulting in only a 34 percent deployment success rate.
- **A2C — Improved Precision with Instability Risks:** Integrating Generalized Advantage Estimation provided A2C with excellent physical precision by smoothing the reward signals. However, lacking a constraint on policy updates, it suffered a **catastrophic collapse** when a single batch of poor data severely altered the network weights.
- **PPO — Optimal Balance of Stability and Precision:** Incorporating the **clipped objective** eliminated the A2C collapse entirely. The clipping mechanism restricts the magnitude of policy changes in any single update. This structure creates an ideal balance of rapid learning, high precision, and absolute training safety.

4.6 Deployment and Physical Observation

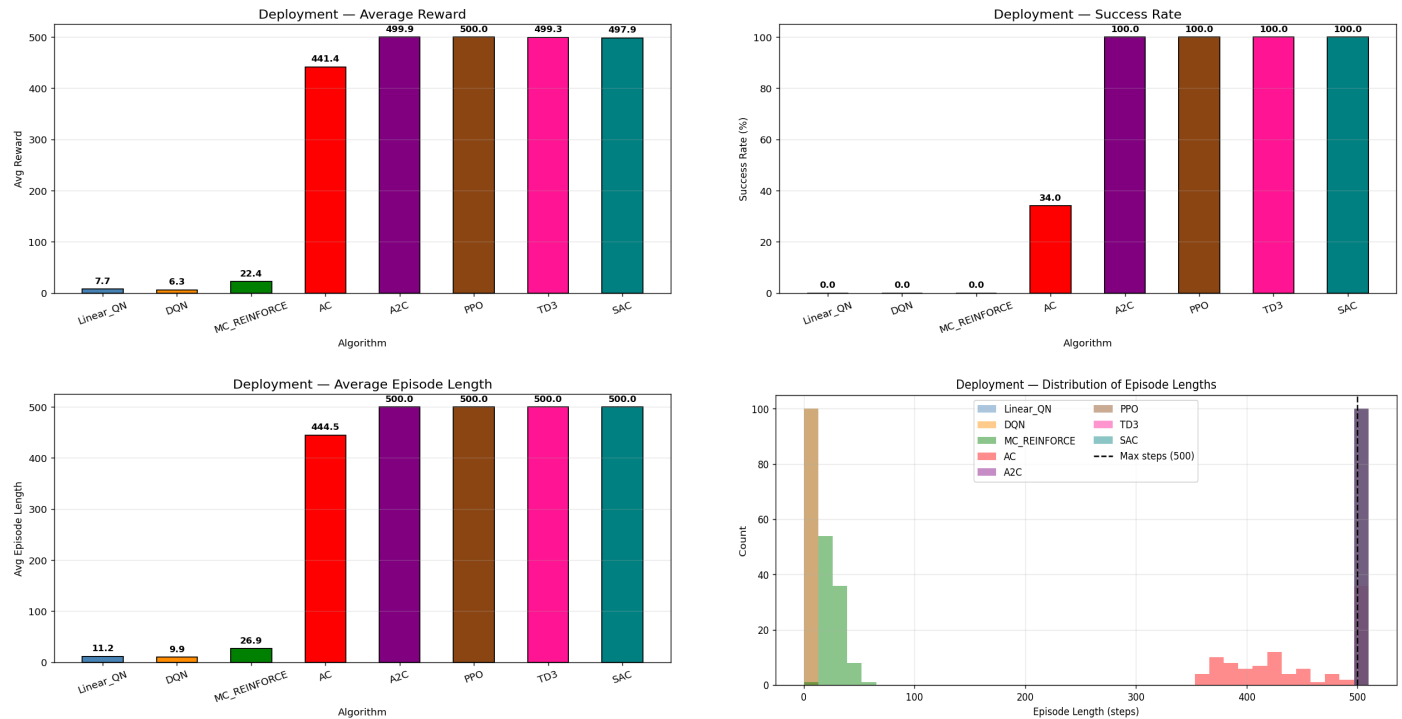


Figure 3: Deployment metrics showing that passing training does not guarantee deployment success.

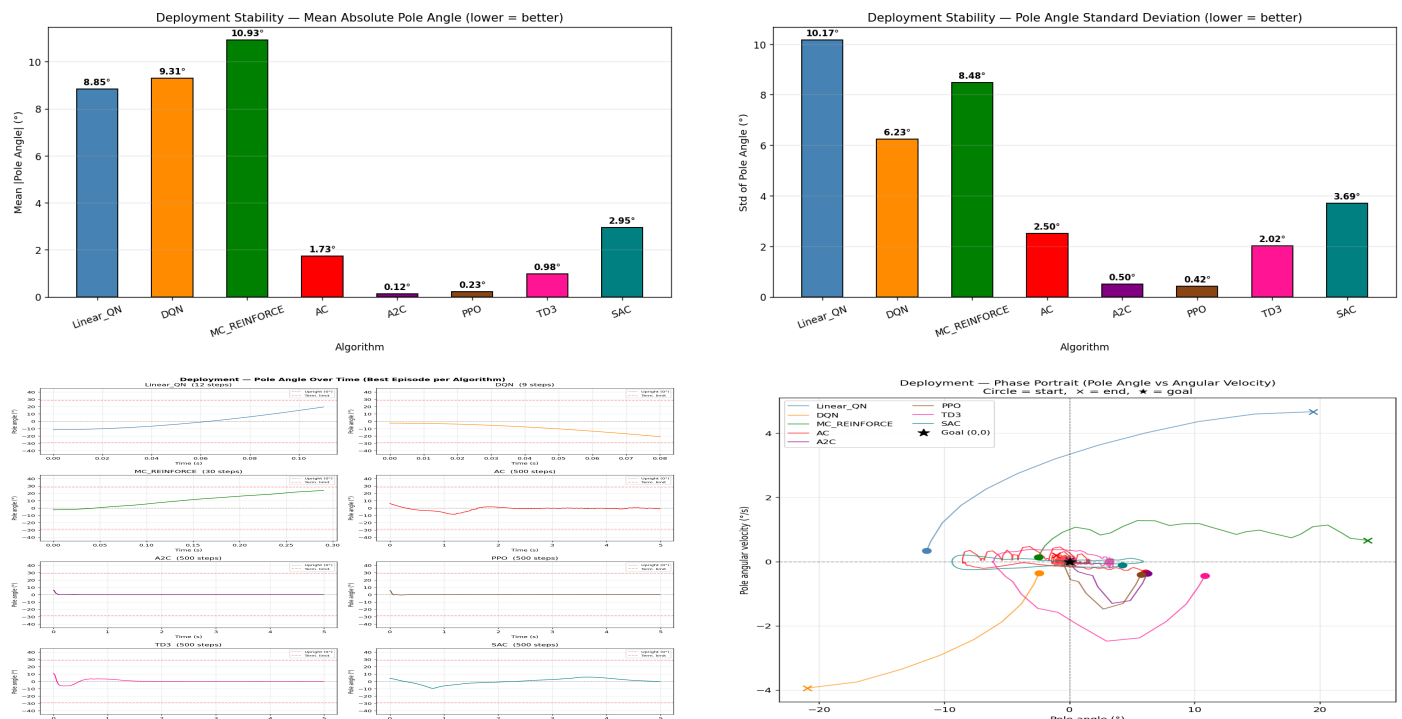


Figure 4: Physical control evaluation showing optimal stabilization for A2C and PPO.

The Training–Deployment Gap: Achieving high scores during training does not automatically translate to reliable real-world performance. **AC passed the training threshold but failed 66 percent of the time during deployment.** This result proves that stochastic exploration noise during training can obscure fundamental flaws in the learned policy. A2C, PPO, TD3, and SAC all achieved 100 percent deployment success, but analyzing their physical control quality reveals critical operational differences.

Action Smoothness and Spatial Control:

- **A2C and PPO — Optimal Control:** These algorithms maintained the pole perfectly upright with an average deviation under 0.25 degrees, while strictly keeping the cart within a 0.1 meter center boundary. They executed remarkably smooth and proportional forces, behaving identically to highly tuned linear-quadratic regulators.
- **TD3 — Aggressive High-Frequency Control:** It successfully stabilized the pole but relied on rapidly alternating forces at the maximum 10 Newton limit. This aggressive control strategy is highly energy inefficient and would cause severe mechanical wear on actual robotic actuators.
- **SAC — High-Entropy Spatial Drift:** It balanced the pole flawlessly but permitted the cart to **drift continuously** toward the 2.5 meter edge of the track. Its entropy-driven objective prioritized action diversity over strict spatial regulation, failing to maintain a stable center position.

Phase-Space Behavior: The phase portrait in Figure 4 provides definitive proof of control stability. A2C and PPO trajectories converge tightly to the exact center origin, indicating absolute physical equilibrium. TD3 generates small continuous loops, confirming its permanent oscillatory state. SAC follows a wide and persistent arc, visually demonstrating its continuous spatial drift across the environment.

Algorithm	Success %	Avg. Reward	Mean $ \theta ^\circ$	Cart Drift	Control Quality
Linear_QN	0	≈ 8	—	—	—
DQN	0	≈ 10	—	—	—
MC REINFORCE	0	≈ 35	—	—	—
AC	34	≈ 170	—	—	Unstable
A2C	100	500	0.12	± 0.1 m	Optimal and Smooth
PPO	100	500	0.23	± 0.1 m	Optimal and Smooth
TD3	100	500	≈ 2.0	± 0.3 m	Aggressive oscillation
SAC	100	500	≈ 8.0	+2.5 m	Continuous drift

Table 5: Deployment physical metrics showing the operational superiority of PPO and A2C.

5 Conclusion

- **PPO is the most effective algorithm.** It converged faster than all alternatives, maintained near-zero variance throughout the training process, and executed flawless physical control during deployment.
- **A2C offers maximum precision but carries high training risks.** It demonstrated slightly tighter pole stabilization than PPO, but the absence of an objective clipping mechanism makes it highly vulnerable to catastrophic training collapse.
- **Discrete-action methods are fundamentally inadequate for this task.** The coarse resolution of discrete force bins prevented delicate balancing, and value-based methods like DQN were ultimately destroyed by the Deadly Triad. Conversely, AC architecture solve this limitation by natively generating precise continuous control signals.
- **A Value Critic is Essential for Policy Gradients:** The complete failure of MC REINFORCE compared to the moderate success of AC proves that a value baseline is strictly necessary to reduce variance and successfully guide policy optimization.
- **The Critical Role of the Clipped Objective:** The primary structural difference between A2C and PPO is the clipped surrogate objective. While A2C experienced severe training crashes, PPO remained perfectly stable. Clipping serves as an indispensable safety constraint for reliable on-policy learning.